

DISOBEY 2027 · WORKING DRAFT

When the Hunter Becomes the Hunted

Deception in Smart Contracts

Ivan Vagunin

THE INVITATION

A target with its own prize pool

Developer

Public instructions.

Verifiable rules.

No reliance on obscurity.

Bug hunter

Low barriers to inspection.

Real assets at stake.

A visible opportunity.

The apparent bug becomes the advertisement.

Three things I need to distinguish

Code

I read

≠

Context

it uses

≠

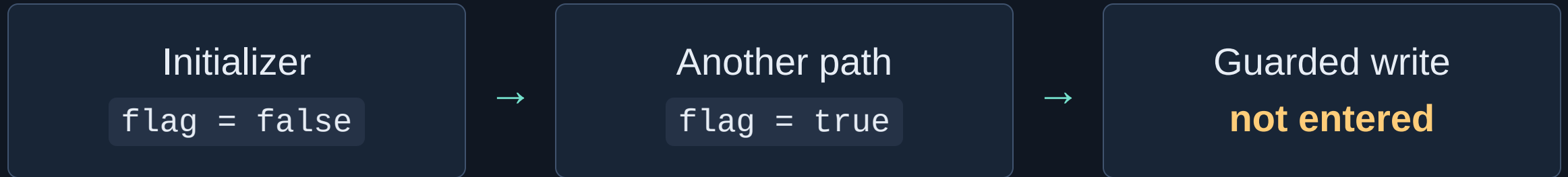
View

I inspect

code + calldata + caller + value
+ state + block environment + call targets

Which missing fact changes the outcome?

The initializer is not the current value



Retrieved source	What it does not establish
Write requires <code>!passHasBeenSet</code> and a value condition	The flag at a selected historical execution
Conditional body can be skipped	That the transaction must revert

Zero value does not mean zero effect

Archived transaction

3 Nov 2017 · SetPass

Block 4484437 · position 13

1 ETH · status: **Success**

Did the guarded write execute?

State evidence still missing.

S1's reported mechanism

Containing transaction

↳ internal call, value = 0

↳ storage changes

Historical filtering can omit the cause.

Receipt status ≠ event list ≠ call trace ≠ state diff

Reachable somewhere $\neq$ reachable here

Symbolic claim

There exists a state in which the guarded write is reachable.

Historical claim

That state is the pre-state of this particular transaction.

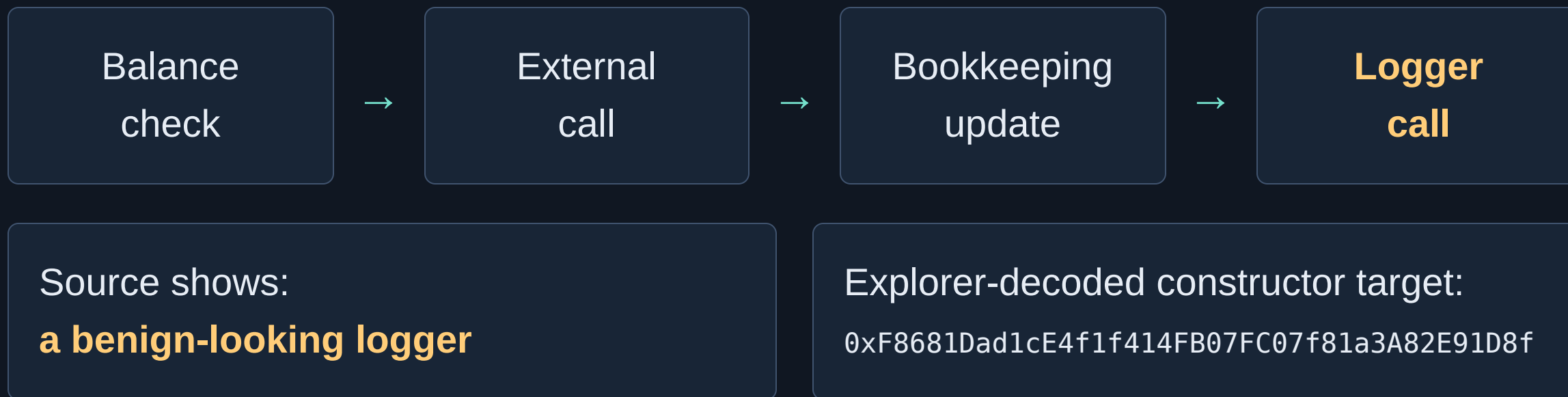
Block boundary

$\neq$

Intermediate transaction pre-state

Record: chain · block hash · transaction position · relevant state

Private Bank: the pattern is not the outcome



A source-level type is not runtime implementation identity.

The decisive frame comes later

Earlier transaction

Deposit commits.

It is **not rolled back** by failure in a later transaction.

Later execution

Intermediate call succeeds.

↓

Dependency fails.

↓

Enclosing frame may fail.

Intermediate success is not final commitment.

Our 15 Feb 2018 containing-transaction record says **Success**.

It is not evidence of the failed path illustrated here.

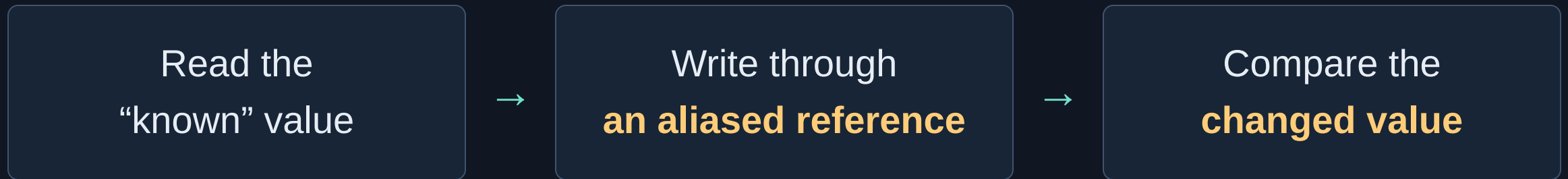
What did the depositor actually send?

Convenient representation	Recovered detail
Figure 9: deposit condition uses <code>>=</code>	Explorer source uses strict <code>></code>
Address row: <code>1.000000 ETH</code>	Transaction: <code>1.0000000623552385 ETH</code>
A logger printed beside the bank	A specific constructor-supplied address

The exact value is above the threshold.

A changed condition, rounded input or substituted dependency changes the explanation.

The write happens before the comparison



```
source reference ┌  
                  └─→ same storage location  
comparison input ┌
```

private is visibility control—not encrypted storage.

Creation → play → destruction



Explorer status “Success” does not prove a payout.

Comparison ← storage location ← prior write

The historical write/read relationship still needs execution-level corroboration.

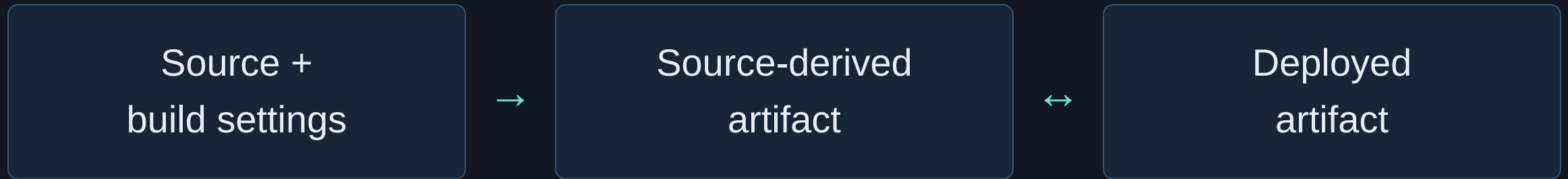
Same appearance. Different string data.

BCT VS BCT

Inspected value	Middle character	Raw UTF-8
ASCII control BCT	U+0043 · Latin C	42 43 54
Author literal BCT	U+0421 · Cyrillic Es	42 d0 a1 54

The helper hashes `abi.encode` of strings; raw UTF-8 is not the complete hash input.
This artifact differs from the paper's `BT` / mutable-symbol illustration.

Which program did we actually analyse?



Historical comparison failure:

executable bytes treated as ignorable metadata.

Author-reported Ropsten call result: stack underflow.

A skipped behavioral difference invalidates the association.

Three meanings of “it looked the same”

Appearance	Unjustified equivalence	Evidence to inspect
Similar rendering	Same data	Encoded predicate inputs
Compatible interface	Same implementation	Actual target and code
Verified source display	Correct artifact association	Artifacts and comparison rules

Familiar appearance encourages an identity assumption.

“Approval For All”: follow the write

```
function approveForAll(address [] calldata _addresses) external onlyOwner {  
    for (uint256 i = 0; i < _addresses.length; i++) {  
        _snapshot[_addresses[i]] = true;  
        emit Approval(_addresses[i], address(this), balanceOf(_addresses[i]));  
    }  
}
```

```
        _balances[to] += amount;  
    }  
    if (_snapshot[from] || _snapshot[to]) require(_snapshotApplied == true, "");
```

Owner-restricted write → either endpoint listed → require the flag

Images: write and condition. Report prose: flag is fixed false.

Full-contract invariant and unique deployment identity remain unresolved.

A successful transaction is not a specification

A comparison must identify	Why it matters
Account and entry point	Different predicates may apply
Relevant pre-state	Membership and flags can differ
Dependencies and execution context	Same source is not the same system
State effect—not only receipt status	Success does not prove the intended write

Different context can explain different results
without a broken simulator.

Revisit the evidence—not the labels

Case	Evidence boundary
Hidden state	Transaction status $\neq$ state postcondition
Private Bank	Displayed source / rounded value $\neq$ execution inputs
Storage reference	Initial value $\neq$ value after preceding writes
Homograph	Rendered string $\neq$ encoded data
Verification	Source display $\neq$ justified artifact association
Token restriction	Familiar name $\neq$ write/read behavior
CPIMP	Expected output $\neq$ expected implementation path

Old mechanisms persist. New workflows and combinations emerge.

WHEN THE HUNTER BECOMES THE HUNTED

“I recognised the bug.”

Did I identify the execution?

Which execution fact would prove
my interpretation wrong?

Questions

Which execution fact would prove my interpretation wrong?

Disobey 2027 · Public materials URL TBD

References · historical mechanisms

S1 · Torres, Steichen, State

The Art of The Scam: Demystifying Honeypots in Ethereum Smart Contracts. USENIX Security 2019.
Paper · Figures 6, 7, 9; §3.2.3; Appendix A.

S2 · Ivanov et al.

Targeting the Weakest Link: Social Engineering Attacks in Ethereum Smart Contracts. 2021.
Paper · §4.2.1 / Figure 5.

S4 · samczsun

Hiding in Plain Sight. 11 November 2021.
Original write-up.

References · trading and verification research

S3 · CertiK. *Honeypot Scams*. 11 January 2024.

Report · Blacklist discussion and images reproduced on slide 15.

S5 · Ma et al. *Abusing the Ethereum Smart Contract Verification Services for Fun and Profit*. NDSS 2024.
Paper.

S6 · Gan, Wang, Lin. *Why Trick Me: The Honeypot Traps on Decentralized Exchanges*. DeFi Security workshop, 2023.
Paper.

CPIMP · primary report and related presentation

P2 · Dedaub team. *The CPIMP Attack.* 15 July 2025.

First-hand response report · Source of slide 17's trace image.

P1 · Peter Kacherginsky. BlockThreat Week 28, 2025.

Secondary chronology; do not merge Kinto's timeline with KIP's trace.

Владислав Азерский · Николай Микрюков

Горшочек, не вари! Истории про Web3-ловушки

ZeroNights, late **2025** · recording published 2026.

Appendix · published deployment identifiers

Gift 1 ETH · S1 Appendix A

0xd8993f49f372bb014fb088eabec95cfdc795cbf6

Private Bank · S1 Appendix A

0xd116d1349c1382b0b302086a4e4219ae4f8634ff

CryptoRoulette · S1 Appendix A

0x94602b0e2512ddad62a935763bf1277c973b2758

Retrieved: explorer source/compiler metadata, creation and selected transaction pages.

Still unresolved: independent historical state/code validation and decisive early-case execution paths.

Dossiers and hashes: docs/cases/ · docs/case-materials-manifest.json

Appendix · Gift and Private Bank records

Gift creation · 29 Oct 2017 · block 4453086

0xab8180c3eea86c9702daacb7fbe17368cfa9c313f90a6d341c72797bd738d7e3

Gift SetPass · 3 Nov 2017 · block 4484437 / position 13

0xa29c1875ef712b7b7c812bf2d1fa48e53ef56423a6882a55c51a1342880636da

Bank creation · 12 Feb 2018 · block 5079695

0x1bd6347bc5c6e0b7af095863e4a2713df4ea932757d852b5a181de817d4bd096

Bank deposit · 13 Feb 2018 · exact value 1.0000000623552385 ETH

0x46fd827bbc02ef0ac626c0f850458191c1b8c36cb9c5375194bafa9642f38124

Later containing transaction · 15 Feb 2018 · explorer status Success

0x494a3fa918f12bb63e0de41474bc63e0c02137ae3ae1f3c58f63864c01fee537

Appendix · CryptoRoulette and CPIMP records

CryptoRoulette creation · 19 Feb 2018 · block 5118518

0xefa4e5dc40389240dbb4027db38b351041b75b488d2e0720ee9b05488f2240e1

Play · 20 Feb 2018 · block 5124620 · 0.1 ETH · status Success

0xc109a14f9588baf3e2a6189795e7a8a10b84a4c5a573effb988c1cc986c0a0d7

Reported destruction · 23 Feb 2018 · block 5144265

0xe40affc8f5d2482bcd2f55eef6d65dbb10346017c05ae785d46503c4f2187d37

CPIMP · Ethereum normal-operation trace cited by Dedaub

0x481a0d295102f50e6e6621d86167a09d7826de635bcd7e1b5623f6462f9a4b46

A transaction reference is not proof of a payout, complete storage effect or victim intent.